



Advanced Web Programming

Ung Văn Giàu
Email: giau.ung@eiu.edu.vn



Unit Testing with Jest and React Testing Library

Topic

We'll cover the following topics:

- Testing pure functions
- Testing components
- Simulating user interactions



Technical requirements

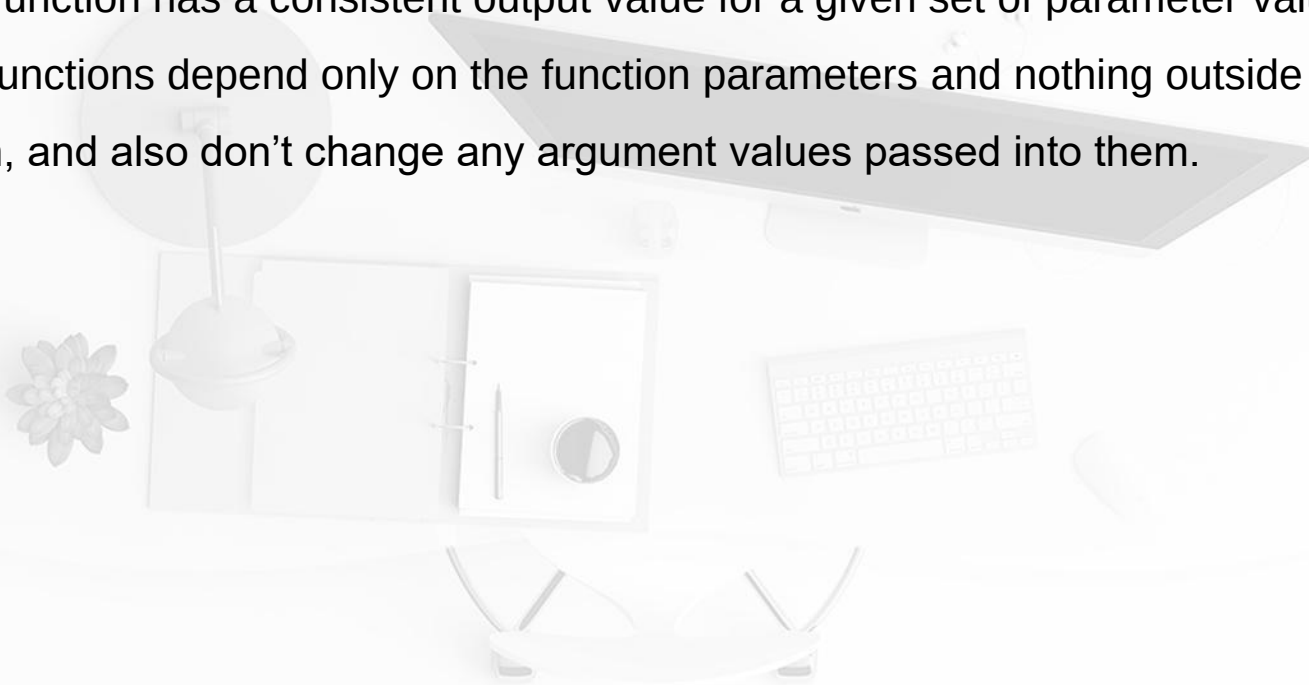
- Node.js and npm
- Visual Studio Code

We will start with a modified version of the code we finished in the last chapter.

- Download the code from EIU Moodle and install all dependencies.

Testing pure functions

- A pure function has a consistent output value for a given set of parameter values.
- These functions depend only on the function parameters and nothing outside the function, and also don't change any argument values passed into them.



Testing pure functions

Understanding a Jest test

- Jest is a delightful JavaScript Testing Framework with a focus on simplicity.
- It works with projects using: Babel, TypeScript, Node, React, Angular, Vue and more.
- Jest is not fully supported by vite due to how the plugin system from vite works.
- These file extensions are **.test.ts** for tests on pure functions and **.test.tsx** for tests on components. Alternatively, a **.spec.*** file extension could be used.

Testing pure functions

Understanding a Jest test

- A test is defined using Jest's **test** function:

```
test('your test name', () => {  
  // your test implementation  
});
```

The test function has two parameters for the **test name** and **implementation**. It is common practice for the test implementation to be an anonymous function.

Testing pure functions

Understanding a Jest test

- The test implementation can be asynchronous by placing the **async** keyword in front of the anonymous function:

```
test('your test name', async () => {  
  // your test implementation  
});
```


Testing pure functions

Understanding a Jest test

- The test implementation will consist of calling the function with arguments being tested and checking the result is as we expect:

```
test('your test name', async () => {  
    const someResult = yourFunction('someArgument');  
    expect(someResult).toBe('something');  
});
```

Jest's **expect** function is used to define our expectations.

Testing pure functions

Understanding a Jest test

- The result of the function call is passed into `expect`, and it returns an object containing methods we can use to define specific expectations for the result. These methods are referred to as **matchers**. If the expectation fails, Jest will fail the test.
- Common matchers:
 - **toBe** matcher checks that **primitive values** are equal.
 - **toStrictEqual** for checking the values in an **object or array**. This recursively checks every property in the object or array.

```
expect(someResult).toStrictEqual({  
  field1: 'something',  
  field2: 'something else'  
});
```

Testing pure functions

Understanding a Jest test

- Common matchers:

- **not** for checking the **opposite** of a matcher.

```
expect(someResult).not.toBe('something');
```

- **toMatch** for checking strings against **regular expressions** (regexes).

```
expect(someResult).toMatch(/error/);
```

- **toContain** for checking if an **element** is **in an array**.

```
expect(someResult).toContain(99);
```

Testing pure functions

Setting up Jest

- Install:
 - `npm install jest --save-dev`
 - `npm install @testing-library/react --save-dev`
 - `npm install ts-jest @types/jest --save-dev`
 - `npm install ts-node @testing-library/jest-dom --save-dev`
 - `npm install jest-environment-jsdom`

Testing pure functions

Setting up Jest

- Configuration:

- In the **package.json**, add **"test": jest** to the **script**

```
"scripts": {  
  "dev": "vite",  
  "build": "tsc -b && vite build",  
  "lint": "eslint .",  
  "preview": "vite preview",  
  "test": "jest"  
},
```

- In your root directory, create a **jest.config.ts** file and add the following:

```
export default {  
  preset: 'ts-jest',  
  testEnvironment: 'jest-environment-jsdom',  
  transform: {  
    "^.+\\.tsx?$": "ts-jest"  
    // process `*.tsx` files with `ts-jest`  
  }  
}
```

Testing pure functions

Setting up Jest

- Configuration:

- In the **src** folder, create a test file called App.test.ts
- Add a simple test in the test file: App.test.ts

```
test('demo', () => {  
  expect(true).toBe(true)  
})
```

- Run the **npm run test** and check results

Testing pure functions

Testing `isChecked`

- The first function we will test is `isChecked`. This function has two parameters:
 - **`checkedIds`**: This is an array of IDs that are currently checked
 - **`idValue`**: This is the ID to determine whether it is checked
- Write a test for when the list item is checked and another for when it isn't checked:
 - Create a file called **`isChecked.test.ts`** in the **`src/Checklist`** folder that will contain the tests.
 - Open **`isChecked.test.ts`** and import the **`isChecked`** function:

```
import { isChecked } from './isChecked';
```

Testing pure functions

Testing isChecked

- Write a test for when the list item is checked and another for when it isn't checked:
 - Create the first test:

```
test('should return true when in checkedIds', () => {  
  );
```

- Forming a **naming convention** for test names is good practice so that they are consistent and easy to understand. Here, we have used the following naming structure:
should {expected output / behaviour} when {input / state condition}

Testing pure functions

Testing isChecked

- Write a test for when the list item is checked and another for when it isn't checked:
 - Implement the logic inside the test:

```
test('should return true when in checkedIds', () => {  
  const result = isChecked([1, 2, 3], 2);  
  expect(result).toBe(true);  
});
```

Testing pure functions

Testing isChecked

- Write a test for when the list item is checked and another for when it isn't checked:
 - Add a second test to cover the case when the ID isn't in the checked IDs:

```
test('should return false when not in checkedIds', () =>
{
  const result = isChecked([1, 2, 3], 4);
  expect(result).toBe(false);
});
```

Testing pure functions

Testing exceptions

- Jest has a **toThrow** matcher that can be used to check whether an exception has been raised.
- For this to catch exceptions, the function being tested has to be executed inside the expectation

```
test('some test', () => {  
  expect(() => {  
    someAssertionFunction(someValue);  
  }).toThrow('some error message');  
});
```

Testing pure functions

Testing exceptions

- Carry out the following steps:
 - Create a file called **assertValueCanBeRendered.test.ts** in the **src/Checklist** folder for the tests and import the **assertValueCanBeRendered** type assertion function:

```
import { assertValueCanBeRendered } from './assertValueCanBeRendered';
```

Testing pure functions

Testing exceptions

- Carry out the following steps:

- The first test we will add is to check whether an exception is raised when the value isn't a string or number:

```
test('should raise exception when not a string or
number', () => {
  expect(() => {
    assertValueCanBeRendered(
      true
    );
  }).toThrow(
    'value is not a string or a number'
  );
});
```

Testing pure functions

Testing exceptions

- Carry out the following steps:
 - Test whether an exception isn't raised when the value is a string:

```
test('should not raise exception when string', () => {  
  expect(() => {  
    assertValueCanBeRendered(  
      'something'  
    );  
  }).not.toThrow();  
});
```

Testing pure functions

Testing exceptions

- Carry out the following steps:
 - The last test will test an exception isn't raised when the value is a number:

```
test('should not raise exception when number', () => {  
  expect(() => {  
    assertValueCanBeRendered(  
      99  
    );  
  }).not.toThrow();  
});
```

Testing pure functions

Running tests

- Carry out the following steps:
 - Open the terminal and execute the following command: **npm run test**
 - Open **assertValueCanBeRendered.ts** and change the expected error message on the first test

```
test('should raise exception when not a string or  
number', () => {  
  expect(() => {  
    assertValueCanBeRendered(true);  
  }).toThrow('value is not a string or a numberX');  
});
```


Testing pure functions

Running tests

- Jest provides valuable information about the failure that helps us quickly resolve test failures.
- It tells us this:
 - Which test failed
 - What the expected result was, in comparison to the actual result
 - The line in our code where the failure occurred

Testing pure functions

Recap

- Tests are defined using Jest's **test** function.
- Expectations within the test are defined using Jest's **expect** function in combination with one or more matchers.
- The **expect** function argument can be a function that executes the function being tested. This is useful for testing exceptions with the **toThrow** matcher.



Q&A